

CI/CD no MICROSOFT FABRIC

Introdução e Fundamentos

Sidney Cirqueira
Alison Pezzott

Agenda:

- Conteudo -> 19:15 as 20:30 hrs
- Intervalo -> 20:30 as 20:40 hrs
- Perguntas -> 20:40 as 20:50 hrs
- Conteudo -> 20:50 as 21:50 hrs
- Perguntas -> 21:50 as 22:00 hrs

Problema:

- Deployment manual criando ambientes inconsistentes e falta de governança
- Suporte a multiplos itens no Microsoft Fabric
- Diferentes processos de DevOps, alguns itens possuem sub artefatos (ex: tabelas de lakehouse, stored procedures em um Data warehouse)

Solução:

- CI/CD no Microsoft Fabric com Azure DevOps + biblioteca python pyfabricops

Objetivos:

- Conceitos de CI/CD no Microsoft Fabric
- Integração com Git e Estratégias de Branch
- Construir e Implementar CI/CD com Azure DevOps usando código YAML
- Gerenciamento de credenciais e segurança no Fabric
- Utilização de biblioteca python para CI/CD no Microsoft Fabric
- Estratégias de git para desenvolvimento colaborativo
- Solução automatizada de ponta a ponta com cenário real no Fabric

Publico Alvo:

- Arquitetos de Dados
- Arquitetos de Solução
- Engenheiros de Dados e Desenvolvedores
- Engenheiros de DevOps
- Administradores de ambientes PowerBI e Fabric

Pré-Requisitos

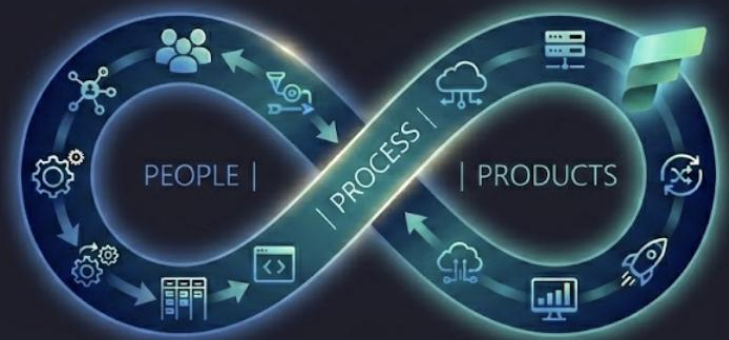
- Subscrição no Azure
- Repositório no Azure DevOps
- Workspace no Fabric com Capacidade ativa
- Azure Key Vault e Service Principal

Conhecimentos Prévios

- Conhecimento básico em Azure
- Conhecimento básico em Git, Branches, PR e Merges
- Familiaridade com YAML
- Entendimento sobre os itens do Microsoft Fabric

O que é DevOps?

“ DevOps é o conjunto de **pessoas, processos e produtos** que habilitam entrega contínua de valores para seus usuários finais ”



CONTINUOUS DELIVERY OF VALUE

O que é Integração Contínua e Entrega Contínua (CI/CD)?

Integração Contínua:

- Mudanças pequenas e incrementais, baseado e equipes de desenvolvimento
- Merge frequente para main branch

Entrega Contínua:

- Entrega de solução em pedaços, ciclos frequentes, confiáveis e processo de deployment repetitivo.

CI/CD no MICROSOFT FABRIC

Fabric Overview

CI/CD no MICROSOFT FABRIC

Configurações de Tenant

+

Configurações GIT

Passa a Passo - Configurações do Azure (EntraID)

Credenciais Service Principal

^ Essentials

Display name : [fabric_app_workshop](#)

Application (client) ID : 4df6a1e1-551b-4c43-a613-b6070dff972f

Object ID : 2d9c25ad-21c7-48d9-a816-cd8d948b5ba4

Directory (tenant) ID : 8b657164-2585-4ff1-a9b8-123681b505ca

Supported account types : [My organization only](#)

Manage

Branding & properties

Authentication (Preview)

Certificates & secrets

Token configuration

API permissions

Expose an API

App roles

Owners

Roles and administrators

Application registration certificates, secrets and federated credentials can be found in the tabs below.

Certificates (0) Client secrets (1) Federated credentials (0)

A secret string that the application uses to prove its identity when requesting a token. Also can be referred to as application password.

+ New client secret

Description	Expires	Value ⓘ	Secret ID
fabric_app_workshop	8/30/2026	D4F*****	69729474-26b5-4637-b94c-a99822a940f3

Passa a Passo - Configurações do Azure (EntraID)

Permissionamento Service Principal

fabric_app_workshop | API permissions ✨ ...

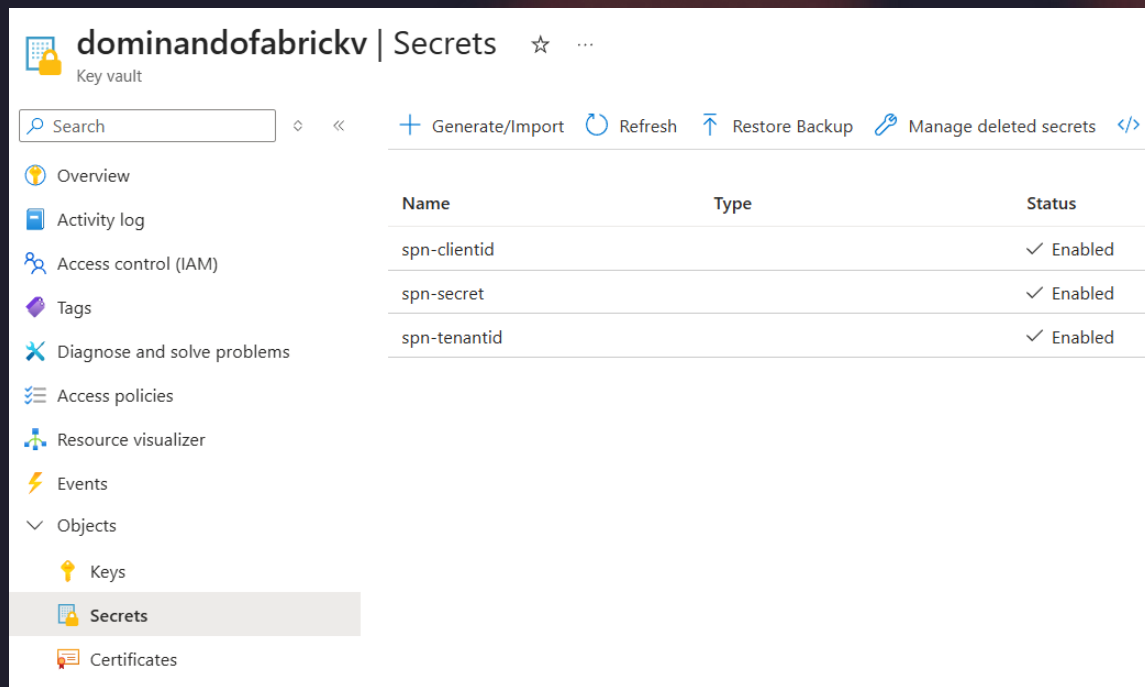
Search ◊ << Refresh Got feedback?

- Overview
- Quickstart
- Integration assistant
- Diagnose and solve problems
- Manage
 - Branding & properties
 - Authentication (Preview)
 - Certificates & secrets
 - Token configuration
 - API permissions**
 - Expose an API
 - App roles
 - Owners
 - Roles and administrators
 - Manifest
- Support + Troubleshooting

API / Permissions name	Type	Description	Admin consent requ...	Status
▼ Azure DevOps (2) ...				
user_impersonation	Delegated	Have full access to Visual Studio Team Services REST APIs	No	✔ Granted for Data Vision ...
vso.pipelinesources_use	Delegated	Pipeline Resources (use)	No	✔ Granted for Data Vision ...
▼ Microsoft Graph (1) ...				
User.Read	Delegated	Sign in and read user profile	No	✔ Granted for Data Vision ...
▼ Power BI Service (8) ...				
Item.Execute.All	Delegated	Make API calls that require execute permissions on all Fab...	No	✔ Granted for Data Vision ...
Item.Read.All	Delegated	Make API calls that require read permissions on all Fabric i...	No	✔ Granted for Data Vision ...
ItemMetadata.Read.All	Delegated	Make API calls that can read item metadata for all Fabric it...	No	✔ Granted for Data Vision ...
Pipeline.Deploy	Delegated	Deploy in all pipelines	No	✔ Granted for Data Vision ...
Pipeline.Read.All	Delegated	View all pipelines	No	✔ Granted for Data Vision ...
Workspace.GitCommit.All	Delegated	Make API calls that commit workspace content and settin...	No	✔ Granted for Data Vision ...
Workspace.GitUpdate.All	Delegated	Make API calls that update workspace content and setting...	No	✔ Granted for Data Vision ...
Workspace.Read.All	Delegated	View all workspaces	No	✔ Granted for Data Vision ...

Passa a Passo - Configurações do Azure (EntraID)

Azure Key Vault



The screenshot shows the Azure Key Vault interface for a vault named 'dominandofabrickv'. The 'Secrets' tab is selected, displaying a list of secrets. The left sidebar contains navigation options: Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Access policies, Resource visualizer, Events, Objects, Keys, Secrets (highlighted), and Certificates. The top of the Secrets page includes a search bar and action buttons: Generate/Import, Refresh, Restore Backup, and Manage deleted secrets. The secrets are listed in a table with columns for Name, Type, and Status.

Name	Type	Status
spn-clientid		✓ Enabled
spn-secret		✓ Enabled
spn-tenantid		✓ Enabled

Passa a Passo - Configurações do Azure (EntraID)

Permissionamento Key Vault

Home > Key vaults > dominandofabrickv

dominandofabrickv | Access control (IAM) ☆ ...

Key vault

Search

+ Add Download role assignments Edit columns Refresh Delete Feedback

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Access policies

Resource visualizer

Events

Objects

Keys

Secrets

Certificates

Settings

Monitoring

Automation

Help

Key Vault Contributor (3)

☐	 fabric_app_workshop 4df6a1e1-551b-4c43-a613...	Service principal	Key Vault Contributor	 This resource	None
☐	 User1 65998eb9-8bfe-434c-9522-...	User	Key Vault Contributor	 This resource	None
☐	 workshop_user_managed 6a1b1a66-4fc4-4a22-8bab...	Managed identity	Key Vault Contributor	 This resource	None

Key Vault Reader (4)

☐	 Alison Pezzott alison.pezzott@overdax.com	User	Key Vault Reader	 This resource	None
☐	 fabric_app_workshop 4df6a1e1-551b-4c43-a613...	Service principal	Key Vault Reader	 This resource	None
☐	 Sidney Cirqueira sidneyocirqueira@sidneyoc...	User	Key Vault Reader	 This resource	None
☐	 sidneyoliveiracirqueira-Wo... 0c10b2c3-ee76-4184-b345...	Service principal	Key Vault Reader	 This resource	None

Key Vault Secrets Officer (1)

Key Vault Secrets User (4)

☐	 Alison Pezzott alison.pezzott@overdax.com	User	Key Vault Secrets User	 This resource	None
☐	 Alison Pezzott User alison.pezzott@outlook.com	User	Key Vault Secrets User	 This resource	None
☐	 fabric_app_workshop 4df6a1e1-551b-4c43-a613...	Service principal	Key Vault Secrets User	 This resource	None

Passa a Passo - Configurações Microsoft Fabric

Conexões Keyvault

The screenshot displays the 'Manage Connections and Gateways' interface in Microsoft Fabric. The main area shows a table of connections, with the 'conn_keyvault' connection highlighted. A red box highlights the 'Users' column for this connection, showing 'Alison Pezzott, Sidney'. The right sidebar shows the 'Settings' for this connection, including the 'Reference alias' (conn_keyvault), 'Connection ID' (91ed0fb7-75e5-4175-8281-718efb808808), 'Connection type' (Azure Keyvault Service), and 'Data source path' (dominandofabrickv). The 'Authentication' section shows 'Authentication method' set to 'OAuth 2.0' with an 'Edit credentials' link. The 'General' section has a checkbox 'Allow this Azure Key Vault reference to be utilized with either on-premises data gateways or VNet data gateways' which is checked.

Power BI Workshop_DMF-DEV

Trials activated: 59 days left

Settings Manage users Remove Get help

Manage Connections and Gateways

Connections On-premises data gateways Virtual network data gateways Streaming virtual network data gateways Azure Key Vault references

The Azure Key Vault reference provides a secure way to retrieve credentials from your Azure Key Vault resource. [Learn more in this overview.](#)

Name ↑	Users	Status
✓ conn_keyvault	Alison Pezzott, Sidney	🔄

Settings

Reference alias *

conn_keyvault

Connection ID

91ed0fb7-75e5-4175-8281-718efb808808

Connection type

Azure Keyvault Service

Data source path

dominandofabrickv

Authentication

Authentication method *

OAuth 2.0

☒ [Edit credentials](#)

☐ Allow Code-First Artifacts like Notebooks to access this connection (Preview)

General

☒ Allow this Azure Key Vault reference to be utilized with either on-premises data gateways or VNet data gateways

Passa a Passo - Configurações do Tenant

Admin portal

Tenant settings

New

Users

Premium Per User

Audit logs

Domains

Workloads

Tags

Capacity settings

Refresh summary

Embed Codes

Organizational visuals

Organizational themes (preview)

ⓘ There are new or updated tenant settings. Expand to review the changes.



Git integration

- Users can synchronize workspace items with their Git repositories
Enabled for the entire organization
- Users can export items to Git repositories in other geographical locations
Disabled for the entire organization
- Users can export workspace items with applied sensitivity labels to Git repositories
Enabled for the entire organization
- Users can sync workspace items with GitHub repositories
Enabled for the entire organization

Funcionalidades de integração com o git podem ser habilitadas pelo administrador do tenant, administrador da capacidade ou administrador do workspace.

Passa a Passo - Configurações do Tenant

Admin **API** settings

- Service principals can access read-only admin **APIs**
Enabled for a subset of the organization
- Service principals can access admin **APIs** used for updates
Enabled for a subset of the organization
- Enhance admin **APIs** responses with detailed metadata
Enabled for a subset of the organization
- Enhance admin **APIs** responses with DAX and mashup expressions
Disabled for the entire organization

Funcionalidades de administração de API podem ser habilitadas pelo administrador do tenant, administrador da capacidade ou administrador do workspace.

Passa a Passo - Integração GIT

- Logar na organização do Azure DevOps (<https://aex.dev.azure.com/>)
- Criar Projeto no Azure DevOps
- Inicializar o REPO no Azure DevOps
- Configurar conexão entre o Fabric Workspace e Git
- Sincronizar os itens para o Repositório

Git

Introdução ao **versionamento** e
desenvolvimento colaborativo

1. Fundamentos de versionamento

Você já fez isso?

projeto_final.zip

projeto_final_v2.zip

projeto_final_agora_vai.zip

Esse é o versionamento
manual.

Ele funciona até...

não funcionar mais:

Impossível saber o que mudou entre versões

Impossível trabalhar em equipe sem
sobrescrever o trabalho do outro

Impossível voltar para um ponto específico do
passado com segurança

Versionamento de código resolve **tudo**
isso de forma sistemática.

O que é git?

O que é git?

É um sistema de controle de versão distribuído, criado em 2005 por Linus Torvalds (o criador do Linux).

O kernel do Linux usava um VCS proprietário chamado BitKeeper. Quando a licença gratuita foi revogada, Linus decidiu criar o próprio em 2 semanas. O resultado foi o Git.



Git ≠ GitHub

Essa confusão é muito comum. Veja a diferença



	Git	GitHub
O que é	Sistema de versionamento de código	Plataforma Web que usa Git
Onde roda	No computador	Na nuvem (Microsoft)
Função	Versionar código	Hospedar repositórios Git + Colaboração
Alternativas	Mercurial, SVN	Azure DevOps, Gitlab, Bitbucket

1. Configurando o ambiente

Windows

1. Acesse <https://git-scm.com/download/win>
2. Baixe o instalador e execute
3. Opções recomendadas no instalador:
 - Default editor: VS Code (ou o editor de sua preferência)
 - Default branch name: main
 - PATH: "Git from the command line and also from 3rd-party software"
 - HTTPS backend: OpenSSL
 - Line ending: "Checkout Windows-style, commit Unix-style"

Via Homebrew (recomendado)

macOs

```
brew install git
```

Verificar instalação

```
git --version
```

Linux (Ubuntu/Debian)

```
sudo apt update
```

```
sudo apt install git -y
```

```
git --version
```

Configurações

```
git config --global user.name "Seu Nome"
```

```
git config --global user.email "seu@email.com"
```

```
git config --global init.defaultBranch main
```

```
# VS Code git config --global core.editor "code --wait" # Vim git config --global core.editor "vim" #  
Notepad++ (Windows) git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -  
multInst -notabbar -nosession -noPlugin"
```

1. Conceitos internos

O conceito mais importante

As três áreas do Git

Working Tree (Diretório de trabalho)

- Onde edita os arquivos
- Git enxerga como **modified** ou **untracked**

Staging Area (Área de Preparação / Index)

- Uma área intermediária onde você prepara o que vai entrar no próximo commit
- Pense como um carrinho de compras: você coloca o que quer, revisa, e só então "finaliza o pedido" com o commit
- Permite criar commits cirúrgicos: só o que faz sentido junto

Repository (.git)

- Onde o Git armazena todo o histórico
- Fica na pasta oculta .git na raiz do projeto
- Contém todos os commits, branches, tags e configurações do repositório

Estado dos arquivos

Untracked

Arquivo novo que o Git
ainda não conhece



git add

Staged

Arquivo adicionado ao index,
pronto para o commit



git commit

Committed

Arquivo salvo no repositório
com segurança

git add



Modified

Arquivo rastreado que foi
alterado mas não está no stage



Arquivos editados



O que é um commit?

Um commit é um snapshot (foto) de todos os arquivos rastreados no momento em que ele é criado.

Importante: Git não salva diferenças (diffs). Ele salva o estado completo de cada arquivo. Isso o torna muito rápido para operações como checkout.

1. Comandos essenciais

<code>git init</code>	Inicializa o repositório na pasta atual
<code>git status</code>	Mostra o estado atual do repositório
<code>git add .</code>	Adiciona todos arquivos do diretório atual à staging area.
<code>git add -A</code>	Adiciona todos arquivos do repositório atual à staging area
<code>git add pasta/</code>	Adiciona todos arquivos de uma pasta à staging area
<code>git add arquivo.txt</code>	Adiciona um arquivo específico à staging area
<code>git commit -m "mensagem"</code>	Comita o que está no staging. Cada -m abre nova linha
<code>git commit -am "mensagem"</code>	add + commit (só para arquivos já rastreados)
<code>git commit --amend</code>	modifica o último commit (mensagem ou conteúdo)
<code>git commit --amend --no-edit</code>	emenda o commit sem mudar a mensagem

1. Histórico

<code>git log</code>	Histórico completo
<code>git log --oneline</code>	Uma linha por commit
<code>git log --oneline --graph</code>	Com grafo de branches
<code>git log --oneline --graph --all</code>	Inclui branches remotas
<code>git log --oneline -n 5</code>	Últimos 5 commits
<code>git log --author="João"</code>	Filtra por autor
<code>git log --since="2 weeks ago"</code>	Commits das últimas 2 semanas
<code>git log --grep="fix"</code>	Filtra pela mensagem
<code>git log -- arquivo.txt</code>	Histórico de um arquivo específico
<code>git log p arquivo.txt</code>	Histórico + diff de um arquivo
<code>git log --stat</code>	Mostra quais arquivos foram alterados

<code>git diff</code>	working tree vs staging area
<code>git diff --staged</code>	staging area vs último commit
<code>git diff HEAD</code>	working tree vs último commit
<code>git diff main feature</code>	diferença entre duas branches
<code>git diff a3f8c2d b7e1d4f</code>	diferença entre dois commits
<code>git diff HEAD~1 HEAD</code>	último commit vs o anterior
<code>git diff --stat</code>	resumo: quais arquivos e quantas linhas

<code>git show</code>	Mostra o último commit
<code>git show a3f8c2d</code>	Mostra um commit específico
<code>git show HEAD~2</code>	Mostra 2 commits atrás do HEAD
<code>git show HEAD:arquivo.txt</code>	Mostra o conteúdo de um arquivo em um commit

<code>git restore arquivo.txt</code>	Descarta mudanças no working tree
<code>git restore .</code>	Descarta TODAS as mudanças não staged
<code>git restore --staged arquivo.txt</code>	Remove do stage (mantém as mudanças)
<code>git restore --source=HEAD~2 arquivo.txt</code>	Restaura arquivo de 2 commits atrás

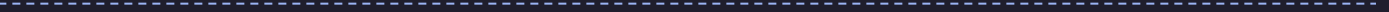
<code>git rm arquivo.txt</code>	Remove do disco e do tracking
<code>git rm --cached arquivo.txt</code>	Remove do tracking mas mantém do disco
<code>git rm -r pasta/</code>	Remove pasta recursivamente

`git rm --cached` é muito útil para remover do Git um arquivo que você acidentalmente commitou (como um `.env`).

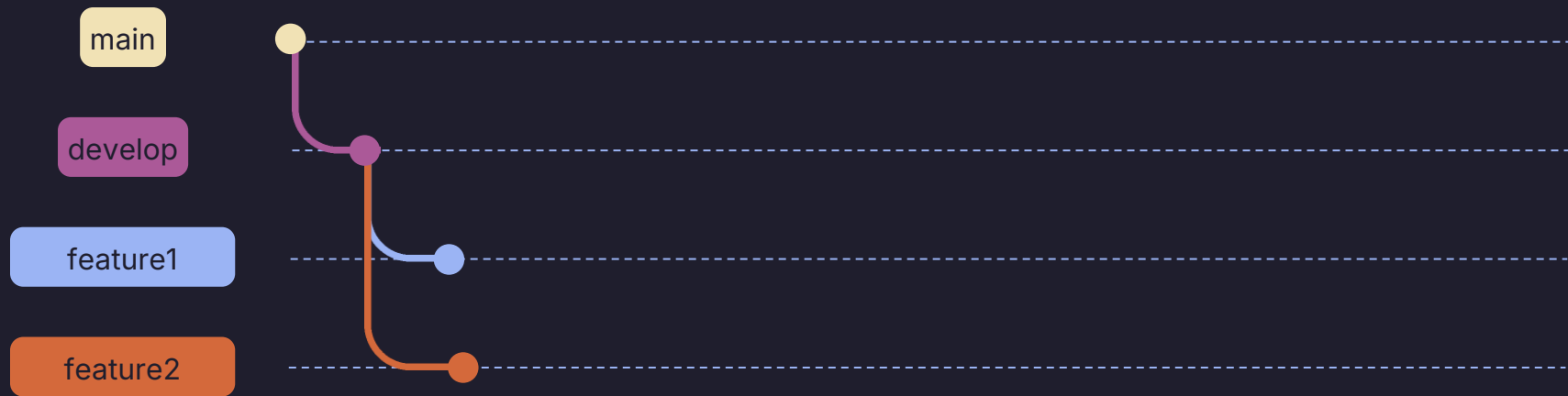
<code>git mv arquivo.txt novo-nome.txt</code>	Renomeia um arquivo
<code>git mv arquivo.txt pasta/arquivo.txt</code>	Move um arquivo

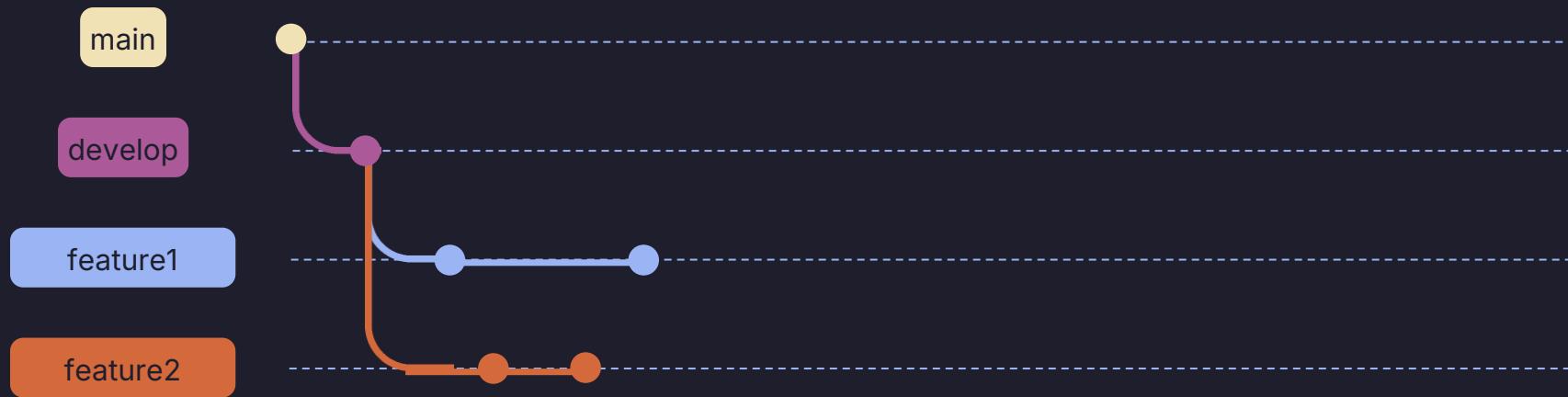
1. Branches

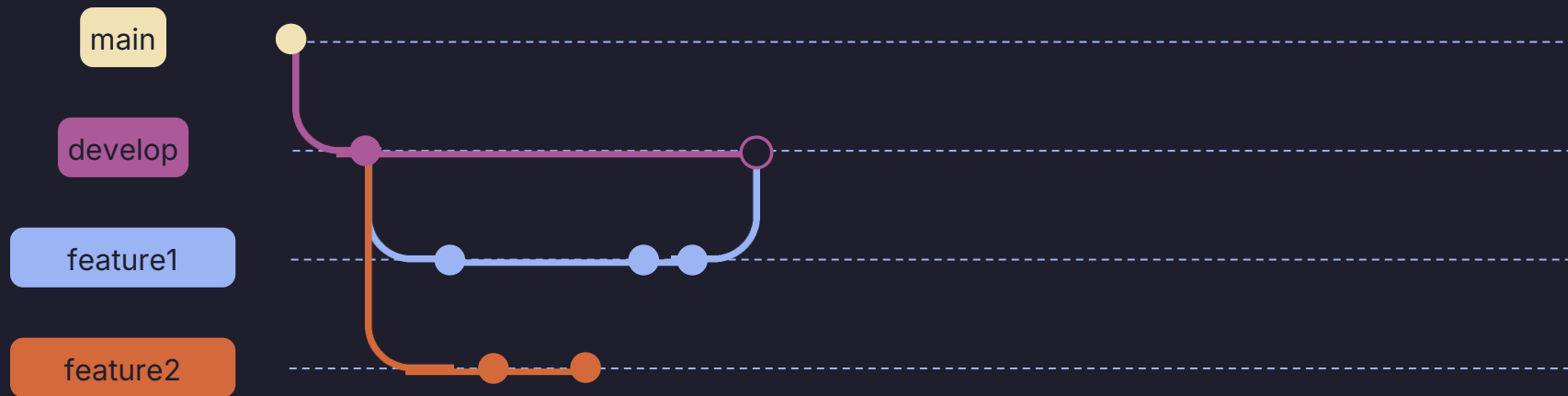
main









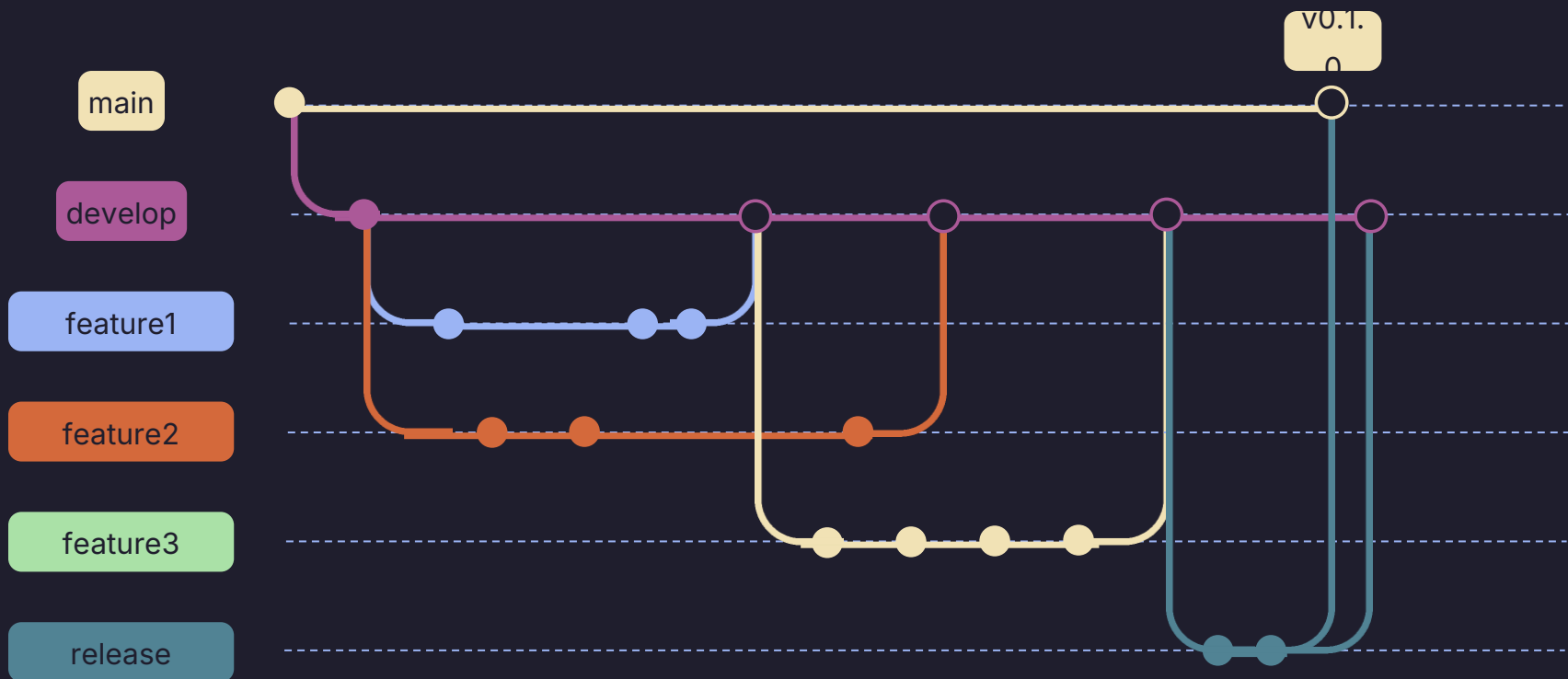


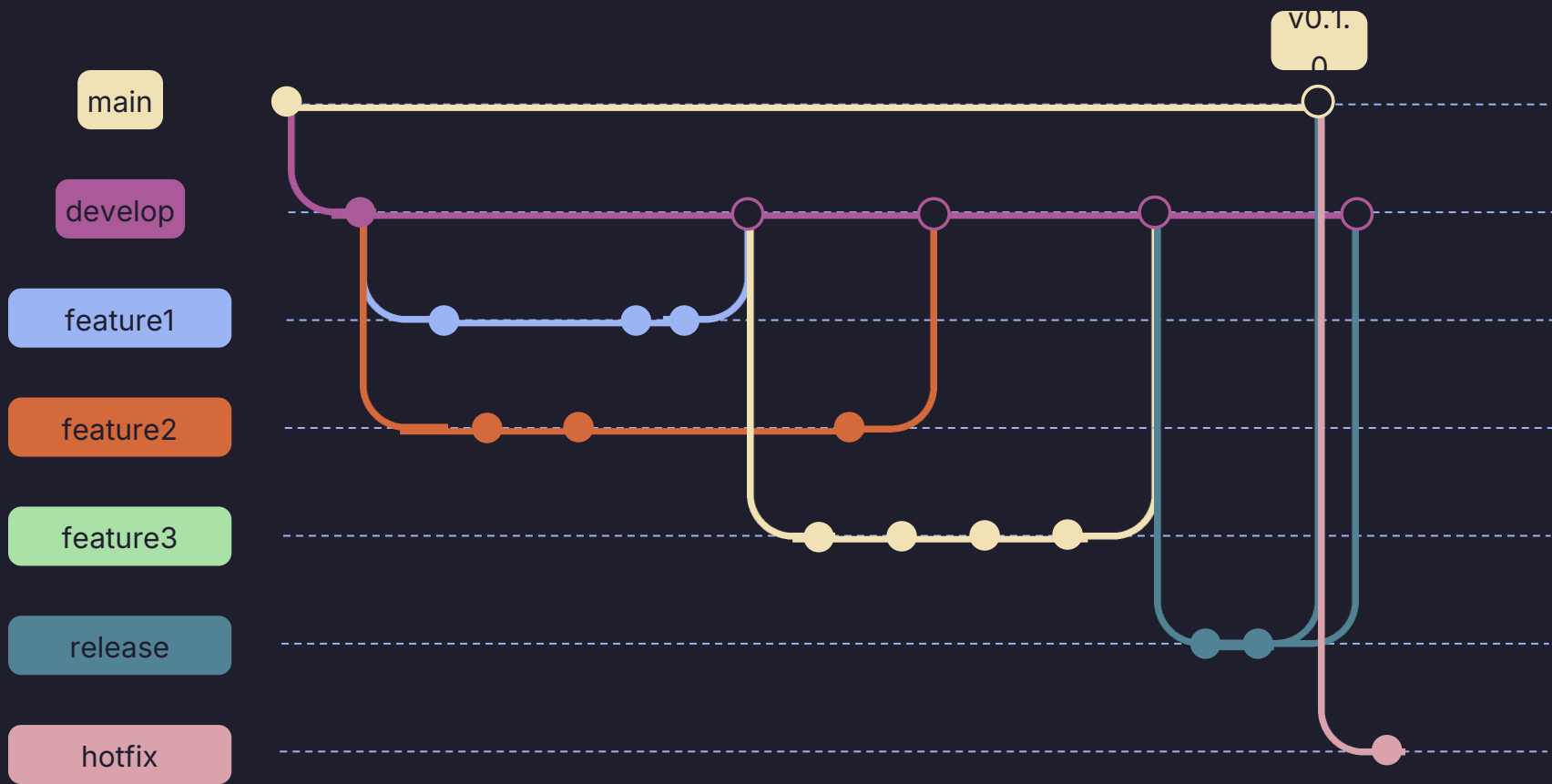


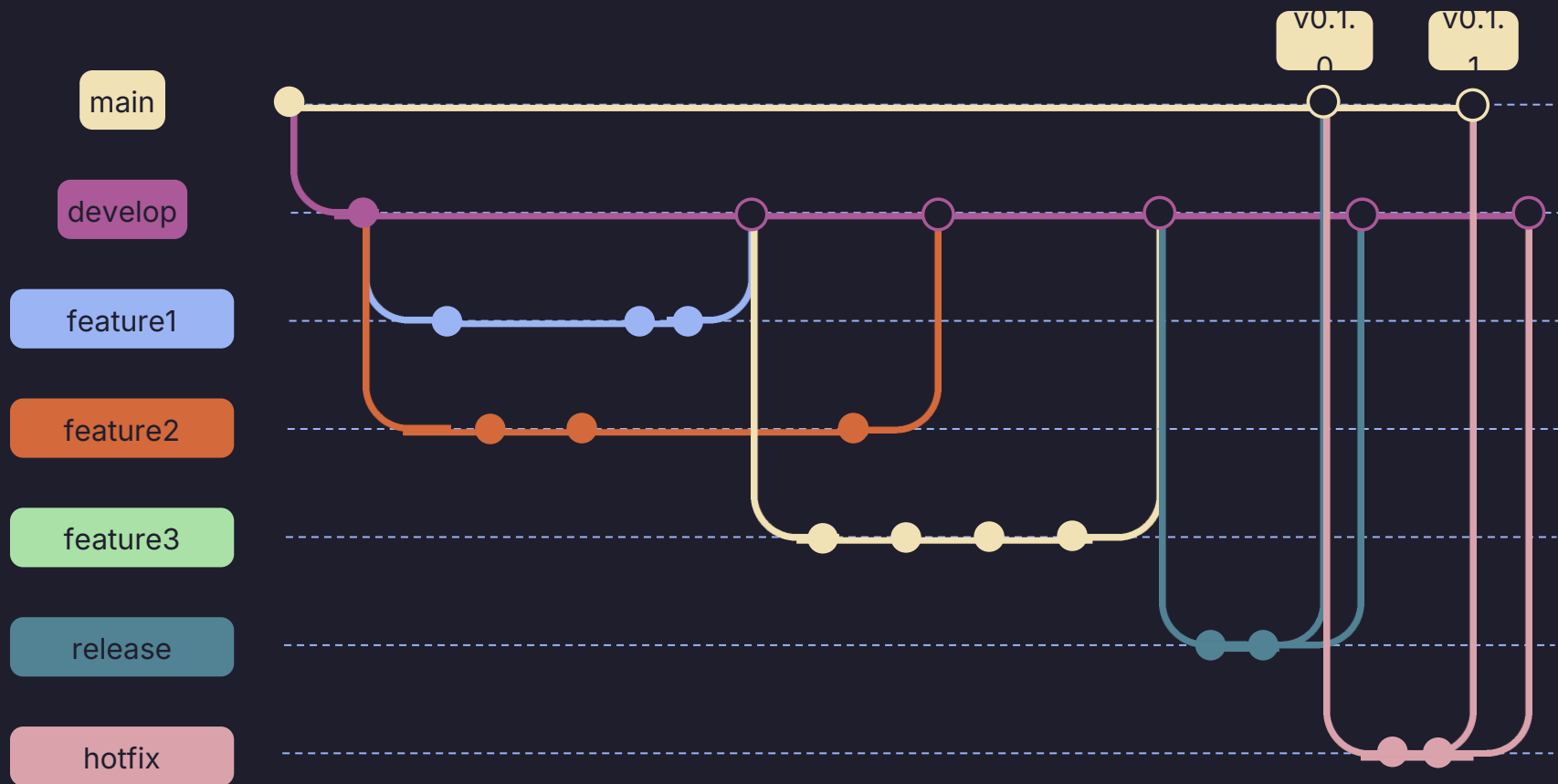






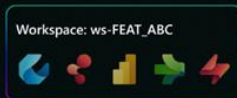






FabricOpsFlow

Features (Temporary branches)



Commit to Git Sync from git



Branch:
feature/ABC



Pull Request

Build and test



- Build
- Scan conflicts
- Verify BPA policies
- Check data quality

Merge



Deploy with
pyFabricOps library



Branch:
develop

Pull Request

Review
and approvals



- Review
- GMUD
- Staging
- Production

Merge



Deploy with
pyFabricOps library



Branch:
main

Clone
Pull



Commit
Push

Repo local feature/ABC



Other features



- Azure Key Vault
- Service principal based
- IAM control
- Synced pipeline variables



- Variable libraries
- Switch by branch
- Branch detected
- Friendly setup



- Azure pipelines (YAML)
- Selective, full or manual deployment



- pyFabricOps lib based
- Local, pipeline or MSFT Fabric notebooks running
- Support for oauth and SPN authentication
- +350 functions

<code>git branch</code>	Lista branches locais
<code>git branch -a</code>	Lista branches locais e remotas
<code>git branch -r</code>	Lista branches remotas
<code>git branch -v</code>	Lista com o último commit de cada
<code>git branch nome-da-branch</code>	Cria branch
<code>git switch nome-da-branch</code>	Muda para branch
<code>git checkout -b nome-da-branch</code>	Cria e muda para branch
<code>git branch -m novo-nome</code>	Renomeia a branch atual
<code>git branch -d nome-da-branch</code>	Deleta a branch (se foi mergeada)
<code>git branch -D nome-da-branch</code>	Força a deleção da branch
<code>git push origin --delete feature/login</code>	Deleta a branch no remoto

1. Merge/Rebase

<code>git merge feature/login</code>	Merge da feature na branch atual
<code>git merge --no-ff feature/login</code>	Força a criação de um merge commit
<code>git merge --squash feature/login</code>	Squash: junta todos os commits em um
<code>git merge --abort</code>	Cancela merge em andamento
<code>git rebase main</code>	Reaplica commits da feature sobre main
<code>git rebase --continue</code>	Continua após resolver conflito
<code>git rebase --abort</code>	Cancela o rebase
<code>git rebase --skip</code>	Pula o commit atual
<code>git rebase -i HEAD~3</code>	Rebase interativo nos últimos 3 commits

Rebase interativo

Uma das ferramentas mais poderosas do Git. Permite reescrever o histórico.

```
git rebase -i HEAD~4  # abre editor com os  
últimos 4 commits
```

```
pick a1b2c3d feat: adiciona header
```

```
pick b2c3d4e fix: corrige typo no header
```

```
pick c3d4e5f feat: adiciona footer
```

```
pick d4e5f6a wip: trabalho em progresso
```

Comando	Ação
pick	Mantém o commit
reword	Mantém o commit mas edita a mensagem
edit	Para no commit para emendas
squash	Junta com o commit anterior
fixup	Junta com o commit anterior (descarta a mensagem)
drop	Remove o commit

Merge x Rebase

	Merge	Rebase
Histórico	Preserva o histórico real	Cria histórico linear
Merge commits	Cria merge commits	Não cria
Segurança	Sempre seguro	⚠ Não em branches públicas
Rastreabilidade	Fácil ver quando branches se uniram	Histórico mais limpo
Quando usar	Branches públicas, features grandes	Branches locais, limpeza de histórico

Resolvendo conflitos

HEAD
Estrutura de um conflito

código da branch atual (main)

=====

código da outra branch (feature)

>>>>>> feature/login

Como resolver

1. Abra o arquivo em conflito
2. Edite o conteúdo — decida qual código manter (ou misture os dois)
3. Remova os marcadores <<<<<<, =====, >>>>>>
4. Adicione ao stage: git add arquivo.txt
5. Finalize: git merge --continue ou git commit

1. Remotos e GitHub

<code>git remote -v</code>	Lista os remotes com suas URLs
<code>git remote add origin https://github.com/usuario/repo.git</code>	adiciona remote
<code>git remote add origin git@github.com:usuario/repo.git</code>	Via SSH
<code>git remote rename origin upstream</code>	Renomeia o remote
<code>git remote remove origin</code>	Remove o remote
<code>git remote set-url origin git@github.com:usuario/repo.git</code>	Altera a URL

origin — o remote principal (de onde você clonou)

upstream — o repositório original em forks

git fetch: Baixa atualizações do remote sem integrar ao seu trabalho local.

<code>git fetch</code>	Busca todas as atualizações do origin
<code>git fetch origin</code>	Explícito
<code>git fetch --all</code>	Busca de todos os remotes
<code>git fetch origin main</code>	Busca só a branch main
<code>git fetch --prune</code>	Remove referências a branches deletadas no remote
<code>git remote set-url origin git@github.com:usuario/repo.git</code>	Altera a URL

git pull: Fetch + merge (ou rebase) em um único comando.

git pull	fetch + merge do remote tracking atual
git pull origin main	Explícito: de origin, branch main
git pull --rebase	fetch + rebase (histórico mais limpo)
git pull --rebase=interactive	fetch + rebase interativo
git pull --ff-only	Só fast-forward; falha se precisar de merge

git push: Envia commits locais para o repositório remoto

<code>git push</code>	Envia a branch atual para o remote configurado
<code>git push origin main</code>	Explícito: de origin, branch main
<code>git push -u origin main</code>	-u define o upstream (só na primeira vez)
<code>git push --all</code>	envia todas as branches
<code>git push --force</code>	⚠ sobrescreve o histórico remoto
<code>git push --force-with-lease</code>	mais seguro: falha se outros fizeram push

<code>git remote -v</code>	Lista os remotes com suas URLs
<code>git remote add origin https://github.com/usuario/repo.git</code>	adiciona remote
<code>git remote add origin git@github.com:usuario/repo.git</code>	Via SSH
<code>git remote rename origin upstream</code>	Renomeia o remote
<code>git remote remove origin</code>	Remove o remote
<code>git remote set-url origin git@github.com:usuario/repo.git</code>	Altera a URL

origin — o remote principal (de onde você clonou)

upstream — o repositório original em forks

Pull Requests

Pipelines